Page Replacement Algorithm Comparator

Project Document · Operating Systems · Java (Swing)

1. Introduction

Virtual memory lets a process use an address space larger than physical memory. The address space is split into fixed-size **pages** and physical memory into **frames** of the same size. Under demand paging, a page is loaded only when it is referenced. Once all frames are full, the operating system must pick a **victim page** to evict whenever a new page is needed. The policy that makes this choice is a *page replacement algorithm*, and its quality is judged by how few page faults it produces.

This project is a Java application that implements three classic policies (FIFO, LRU and Optimal), simulates them step by step, and compares them for any reference string and frame count.

2. Objectives

- Implement the FIFO, LRU and Optimal page replacement algorithms.
- Count page hits, page faults, hit ratio and fault ratio for each.
- Let the user watch each algorithm work one reference at a time.
- Compare the algorithms side by side and explain the differences.
- Provide built-in theory so the tool works as a learning aid.

3. Background

3.1 Key terms

Term	Meaning
Page	Fixed-size block of virtual memory
Frame	Fixed-size block of physical memory holding one page
Reference string	Sequence of page numbers requested by a process

Term	Meaning
Page hit	Requested page is already in a frame
Page fault	Requested page is not in memory and must be loaded
Victim page	Page chosen for eviction
Hit ratio	Hits divided by total references
Fault ratio	Faults divided by total references

3.2 How a page fault is handled

1. The CPU references a page that is not in memory, raising a page-fault trap.
2. The OS checks that the reference is valid; an invalid one terminates the process.
3. If a free frame exists, it is used.
4. Otherwise the replacement algorithm picks a victim (written back to disk first if modified).
5. The needed page is read into the frame and the page table is updated.
6. The interrupted instruction restarts and now finds the page in memory.

3.3 Performance metrics

- Hit Ratio = (Page Hits / Total References) × 100
- Fault Ratio = (Page Faults / Total References) × 100
- Effective Access Time = (1 − p) × ma + p × fault_time, where p is the page-fault rate and ma is the memory access time.

4. Algorithms

	FIFO	LRU	Optimal
Full name	First In, First Out	Least Recently Used	OPT / MIN
Victim chosen	Oldest page in memory	Page unused for the longest time	Page whose next use is farthest ahead (or never)

	FIFO	LRU	Optimal
Future knowledge	No	No	Yes
Data structure	Queue / circular pointer	Recency list, stack or counters	Look-ahead scan
Belady's anomaly	Possible	No	No
Practical in a real OS	Yes, with variants	Yes, approximated	No, benchmark only
Weak spot	Ignores usage; evicts old but busy pages	Cyclic scans larger than memory	Needs the future

FIFO treats the frames as a queue and always evicts the page that arrived first. It is simple and cheap but ignores how often a page is used.

LRU evicts the page that has gone unused the longest, relying on temporal locality. It never suffers from Belady's anomaly but needs recency tracking.

Optimal looks ahead in the reference string and evicts the page used farthest in the future. It gives the minimum possible faults and serves as the benchmark for the others.

For any input, Optimal ⩽ LRU and Optimal ⩽ FIFO in page faults. FIFO and LRU cannot be ranked in general.

5. System Design

5.1 Architecture

The project has two classes in package `os.main.com`, packaged as the Java module `PageReplacementComparator` (requires `java.desktop`).

Component	Role
`PageReplacement`	Holds the three algorithms as static methods, the `Result` and `SimulationStep` data classes, and a console `main`

Component	Role
`PageReplacementGUI`	Swing `JFrame` that calls the algorithms and presents theory, input, simulator and comparator

5.2 Data model

- `Result`: page faults, page hits and the list of `SimulationStep`s.
- `SimulationStep`: step number, requested page, `HIT` or `FAULT`, and a copy of the frame contents after that step (`-1` marks an empty frame).

Each algorithm, `fifo`, `lru` and `optimal`, takes the page array and frame count and returns a `Result`, so the console and GUI share the same logic.

5.3 Algorithm implementation

- **FIFO**: a circular pointer into the frame array. On a fault the page is written at the pointer, which then advances modulo the frame count.
- **LRU**: an `ArrayList` ordered from least to most recently used. A hit moves the page to the end. On a fault with full frames, the first entry is the victim.
- **Optimal**: on a fault with full frames, scan forward for each resident page's next use. A page never used again is replaced immediately; otherwise the page with the farthest next use is replaced.

6. User Interface

The GUI is a single scrolling page with four sections.

1. **Theory**: explanatory cards covering virtual memory, demand paging, page faults, metrics, the three algorithms, a worked example, locality, Belady's anomaly, a comparison matrix, thrashing, the working-set model and key takeaways.
2. **Input**: reference string field, frame spinner (1 to 16), and **Load Example**, **Run Simulation** and **Clear** buttons.
3. **Simulator**: algorithm selector, **First / Previous / Next / Last / Reset** controls, current frame cards, the requested page, a HIT/FAULT badge, and a detailed execution table.
4. **Comparator**: summary cards (references, frames, algorithms) and a results table with hits, faults, hit ratio and fault ratio.

7. Input Validation

- The reference string must not be empty.
- Entries are separated by spaces or commas and must parse as integers.
- Page numbers must be non-negative.
- Frame count must be at least 1 (the GUI limits it to 1 to 16).

Errors appear in a dialog in the GUI or as a message in the console.

8. Worked Example

Reference string `7 0 1 2 0 3 0 4 2 3` with 3 frames (the default input):

Algorithm	Page faults	Page hits	Hit ratio
FIFO	9	1	10%
LRU	8	2	20%
Optimal	6	4	40%

The result follows Optimal (6) ⩽ LRU (8) ⩽ FIFO (9). Optimal evicts pages that are never used again, such as 7 and 1, while FIFO evicts page 0 even though it was just reused.

9. Technology and Setup

- Language: Java, Swing (`javax.swing` , `java.awt`)
- IDE: Eclipse project, configured for JavaSE-25
- Module: `PageReplacementComparator`

Compile and run from the project root:

```
javac -d bin src/module-info.java src/os/main/com/*.java
java -p bin -m PageReplacementComparator/os.main.com.PageReplacementGUI
```

10. Limitations and Future Work

- The tool only compares FIFO, LRU and Optimal. Second Chance (Clock), LFU and NRU could be added.

- Results are tables and step views; charts of faults versus frame count would help show Belady's anomaly.
- Dirty-page write-back cost and effective access time are explained in the theory but not computed from user input.
- There are no automated tests; unit tests for the three algorithms would be a good addition.
- Input is limited to non-negative integer page numbers.

11. Conclusion

The project turns an abstract memory-management topic into something visible. Users can see why Optimal is the lower bound, why LRU usually beats FIFO on programs with locality, and where each policy breaks down, using their own reference strings.

12. References

- Silberschatz, Galvin and Gagne, *Operating System Concepts*, chapter on virtual memory.
- Tanenbaum, *Modern Operating Systems*, chapter on memory management.